

**UNIVERSIDAD NACIONAL AUTÓNOMA DE HONDURAS
FACULTAD DE INGENIERÍA
DEPTO. DE INGENIERÍA EN SISTEMAS**

GUIA DE APRENDISAJE SQL

PARA SQL SERVER

PRESENTADO POR:

RICHARD ANDERZON GARCIA RIVERA

17 DE AGOSTO DEÑ 2025

Tabla de contenido

INDICE DE COMANDOS DDL	4
CREATE	4
ALTER	4
DROP	4
OPERADORES	4
TIPOS DE CONSTRAINTS (RESTRICCIONES)	4
RESTRICCIONES PARA FOREIGN KEY ()	4
OTROS	5
TIPOS DE DATOS EN SQL SERVER	5
INDICE DE COMANDOS DML	6
COMANDOS	6
OPERADORES DML	6
JOINS	6
OPERADORES SQL	6
FUNCIONES	7
FUNCIONES AGREGADAS	7
FUNCIONES DE CADENA	7
FUNCIONES DE FECHA Y HORA	7
FUNCIONES IDENTITY	7
INDICE DE COMANDOS DCL	8
COMANDOS BASICOS	8
COMANDOS TCL	8
CONTROL DE SESIÓN Y CONEXIÓN	8
DEFINICIÓN AVANZADA DE OBJETOS	8
GESTIÓN DE USUARIOS Y ROLES	8
CONTROL AVANZADO DE OBJETOS	8
MANTENIMIENTO Y ADMINISTRACIÓN	8
PERMISOS Y ROLES CON GRANT	9
PERMISOS SOBRE TABLAS	9
PERMISOS SOBRE VISTAS	9
PERMISOS SOBRE BD	9
PERMISOS SOBRE PROCEDIMIENTOS	9
PERMISOS DE ADMINISTRACIÓN	9
COMADOS CURSORES	10
GENERAL	10
CONTROL DE FLUJO ASOCIADO A CURSORES	10
SECUENCIAS	11
GENERAL	11
EJEMPLOS DEL USO DE DDL	12
CREATE	12
DROP	14
ALTER	17
SINTAXIS	17
EJEMPLOS	17
EJEMPLOS DEL USO DE DML	18
SELECT	18
ORDER BY	19
FUNCIONES DE AGREGACION	20
UNION	20
CTE (COMMON TABLE EXPRESSION)	21
SINTAXIS	21
SUBCONSULTAS	22
JOINS	24
INNER JOIN	24
LEFT JOIN	24
RIGHT JOIN	24
CROSS JOIN	25
EJEMPLOS DEL USO DE TCL	26
COMIT AND ROLLBACK	26

GESTIÓN DE USUARIOS	27
CREAR USUARIOS	32
CREAR ROLES	32
AUDITORIA PARA ROLES Y USUARIOS	33
EJEMPLOS	33
ENCRIPCIÓN DE DATOS	34
HAHS	34
CREACION DE ESQUEMAS	35
EJEMPLOS	36
CURSORES	37
SINTAXIS:	37
EXEC	37
BACKUPS	38
FULL BACKUP (Respaldo completo)	38
DIFFERENTIAL BACKUP (Respaldo diferencial)	38
TRANSACTION LOG BACKUP (Respaldo del log de transacciones)	38
COPY-ONLY BACKUP	39
FILE / FILEGROUP BACKUP	39
SECUENCIAS	40
EJEMPLOS	40
TRIGGERS	41
General	41
EJEMPLOS	41
EJERCICIOS	42
AUDITORIA DE BDs	43
METADATOS	45
TEMAS POR CUBRIR	46
PROYECTOS POR CUBRIR	47
Python como aliado en tu camino (explicado en cada punto)	48
Lo que estamos haciendo hoy	49
Módulo 1 – Introducción a SQL Server Agent	49
Módulo 2 – Crear tu primer Job	49
Módulo 3 – Jobs con múltiples Steps	49
Módulo 4 – Monitoreo y Alertas	50
Módulo 5 – Buenas Prácticas	50
Módulo 6 – Proyecto Final	50
REQUISITOS PARA TRABAJOS	51

INDICE DE COMANDOS DDL

CREATE

- ❖ CREATE DATABASE → crea una nueva base de datos.
- ❖ CREATE TABLE → crea una nueva tabla.
- ❖ CREATE VIEW → crea una vista.
- ❖ CREATE INDEX → crea un índice para optimizar búsquedas.
- ❖ CREATE SCHEMA → crea un esquema para organizar objetos.
- ❖ CREATE TRIGGER → crea un disparador asociado a eventos (INSERT, UPDATE, DELETE)

ALTER

- ❖ ALTER DATABASE → cambia configuraciones de la base de datos
- ❖ ALTER TABLE → agrega, modifica o elimina columnas, constraints, claves foráneas, etc.
- ❖ ALTER INDEX → modifica propiedades de un índice (habilitar/deshabilitar)
- ❖ ALTER SCHEMA → Modificar o mover objetos entre esquemas

DROP

- ❖ DROP DATABASE → elimina una base de datos completa
- ❖ DROP TABLE → elimina una tabla (estructura y datos)
- ❖ DROP VIEW → elimina una vista
- ❖ DROP INDEX → elimina un índice.
- ❖ DROP SCHEMA → elimina un esquema

OPERADORES

- ❖ PRIMARY KEY → Define una clave primaria en una tabla.
- ❖ FOREIGN KEY → Define una clave foránea para relacionar tablas.
- ❖ UNIQUE → Garantiza valores únicos en una columna o conjunto de columnas.
- ❖ NOT NULL → Evita que una columna acepte valores nulos.
- ❖ CHECK → Define restricciones de validación en columnas. Se puede usar IN
- ❖ DEFAULT → Define un valor por defecto para una columna.
- ❖ REFERENCES → Especifica la tabla y columna referenciada para claves foráneas.
- ❖ IDENTITY(n,n) → autoincrementar los valores de n en n
- ❖ ADD → usada con alter para agregar características
- ❖ REFERENCES → para referencias otra tabla cuando usamos el FOREIGN KEY

TIPOS DE CONSTRAINTS (RESTRICCIONES)

- ❖ PRIMARY KEY → Clave primaria
- ❖ FOREIGN KEY → Clave foránea
- ❖ UNIQUE → Valores únicos
- ❖ CHECK → Validación de datos
- ❖ DEFAULT → Valor por defecto
- ❖ NOT NULL → No nulos

RESTRICCIONES PARA FOREIGN KEY ()

- ❖ 1. CASCADE Actualiza las claves foránea en los hijos cuando se actualiza la clave primaria del padre
- ❖ 2. SET NULL Establece NULL en la clave foránea de los hijos al actualizar la clave del padre
- ❖ 3. SET DEFAULT Establece el valor por defecto en la clave foránea de los hijos
- ❖ 4. NO ACTION Genera error si se intenta actualizar una clave primaria que está siendo referenciada
- ❖ 5. RESTRICT Similar a NO ACTION pero con verificación inmediata
- ❖ ON DELETE CASCADE : Elimina todos los registros relacionados
- ❖ ON UPDATE CASCADE : actualiza los registros relacionados

OTROS

- ❖ TRUNCATE → Vaciar todos los registros de una tabla sin eliminar su estructura
- ❖ COMMENT → Agregar comentarios descriptivos a objetos
- ❖ RENAME → Renombrar objetos de la base de datos (tablas, columnas)
- ❖ USE name_bd → seleccionar la BD con la que vamos a trabajar

TIPOS DE DATOS EN SQL SERVER

Numéricos

- ❖ INT: Enteros (± 2.147 mil millones)
- ❖ BIGINT: Enteros grandes (± 9.2 trillones)
- ❖ SMALLINT: Enteros pequeños ($\pm 32,767$)
- ❖ TINYINT: Enteros 0-255
- ❖ DECIMAL(p,s): Números exactos (precisión, escala)
- ❖ FLOAT: Números aproximados
- ❖ REAL: Números flotantes simples
- ❖ MONEY: Datos monetarios

Texto

- ❖ VARCHAR(n): Texto variable (1-8000 chars)
- ❖ CHAR(n): Texto fijo (1-8000 chars)
- ❖ TEXT: Texto largo (hasta 2GB)
- ❖ NVARCHAR(n): Texto Unicode variable

Fecha/Hora

- ❖ DATE: Fecha (año, mes, día)
- ❖ TIME: Hora (hora, minuto, segundo)
- ❖ DATETIME: Fecha y hora (1753-9999)
- ❖ DATETIME2 (n): Fecha y hora extendida decimales con n presicion
- ❖ SMALLDATETIME: Fecha y hora compacta
- ❖ SYSUTCDATETIME(): Devuelve la fecha y hora actual del servidor en horario UTC

Binarios

- ❖ BINARY: Datos binarios fijos
- ❖ VARBINARY: Datos binarios variables
- ❖ IMAGE: Datos binarios grandes
- ❖ VARBINARY(MAX) almacena (doc, pdf, imágenes, videos, audio) máx 2 GB por registro
- ❖ VARCHAR(MAX) → almacena texto variable hasta caracteres (2 GB).
- ❖ NVARCHAR(MAX) → almacena texto variable en Unicode hasta $2^{30}-1$ caracteres (1 GB).
- ❖ FILESTREAM: Almacena en sistema de archivos, gestiona con SQL

Otros

- ❖ BIT: Booleanos (0/1)
- ❖ UNIQUEIDENTIFIER: GUID (128 bits)
- ❖ XML: Datos XML
- ❖ JSON: Datos JSON (SQL Server 2016+)

INDICE DE COMANDOS DML

COMANDOS

- ❖ SELECT → Recupera datos de una o varias tablas o vistas.
- ❖ INSERT INTO → Inserta nuevos registros en una tabla.
- ❖ UPDATE → Modifica datos existentes en una tabla.
- ❖ DELETE → Elimina registros específicos de una tabla.
- ❖ MERGE → combina operaciones de INSERT, UPDATE y DELETE en una sola instrucción

OPERADORES DML

- ❖ FROM → Indica la tabla o vista de donde se consultan los datos.
- ❖ ON → Condición de unión entre tablas.
- ❖ USING → Otra forma de indicar la columna de unión.
- ❖ GROUP BY → Agrupar filas por columnas para funciones agregadas.
- ❖ HAVING → Filtrar grupos (después de GROUP BY).
- ❖ ORDER BY → Ordenar resultados.
- ❖ LIMIT / OFFSET / FETCH → Limitar número de filas devueltas.

JOINS

- ❖ INNER JOIN → Devuelve solo las filas que tienen coincidencias en ambas tablas.
- ❖ LEFT JOIN → Devuelve las filas de la tabla izquierda y las coincidencias de la tabla derecha
- ❖ RIGHT JOIN → Devuelve las filas de la tabla derecha y las coincidencias de la tabla izquierda.
- ❖ FULL JOIN → Devuelve ambas tablas, combinando coincidencias, llenado con NULL donde no haya
- ❖ CROSS JOIN → Devuelve el producto cartesiano de ambas tablas
- ❖ SELF JOIN → La tabla se une a sí misma.
- ❖ NATURAL JOIN → Une tablas usando columnas con el mismo nombre y tipo de dato
- ❖ JOIN ... USING → Filas coincidentes usando columna específica

OPERADORES SQL

- ❖ =, <>, !=, <, >, <=, >= → Comparación básica.
- ❖ AND, OR, NOT → Lógica booleana.
- ❖ IN → Comparar con un conjunto de valores.
- ❖ BETWEEN → Rango de valores.
- ❖ LIKE → Comparación con patrones
- ❖ IS NULL / IS NOT NULL → Verificar valores nulos.
- ❖ EXISTS → Verifica existencia de filas en subconsulta.
- ❖ ALL → Retorna TRUE si la condición se cumple para TODOS los valores de la subconsulta
- ❖ VALUES → crusiar para asignar los valores usando INSERT INTO
- ❖ ANY / SOME → Retorna TRUE si la condición se cumple para AL MENOS UNO de los valores

FUNCIONES

FUNCIONES AGREGADAS

- ❖ SUM () → Suma los valores de una columna numérica.
- ❖ AVG () → Calcula el promedio de los valores de una columna.
- ❖ COUNT () → Cuenta el número de filas.
- ❖ MAX () → Obtiene el valor máximo de una columna.
- ❖ MIN () → Obtiene el valor mínimo de una columna.
- ❖ STDDEV ()/VARIANCE () → Desviación estándar o varianza (estadística).
- ❖ GROUP_CONCAT () STRING_AGG () → Concatenar valores de un grupo en una sola cadena

FUNCIONES DE CADENA

- ❖ CONCAT() → Une dos o más cadenas.
- ❖ SUBSTRING()/SUBSTR() → Extrae una parte de una cadena.
- ❖ LENGTH()/LEN() → Longitud de una cadena.
- ❖ UPPER()/LOWER() → Convierte a mayúsculas o minúsculas.
- ❖ TRIM() / LTRIM / RTRIM → Elimina espacios en blanco de una cadena.
- ❖ REPLACE() → Reemplaza un valor dentro de la cadena.

FUNCIONES DE FECHA Y HORA

- ❖ NOW()/CURRENT_TIMESTAMP → Fecha y hora actual
- ❖ GETDATE()
- ❖ DATEFROMPARTS(x, y, z), x= año, y= mes, z=día, crear un valor de tipo DATE
- ❖ CURDATE()/CURRENT_DATE → Solo la fecha actual
- ❖ DATEADD() / ADDDATE() → Sumar días, meses, años a una fecha
- ❖ DATEDIFF() → Diferencia entre dos fechas
- ❖ EXTRACT() → Extrae parte de una fecha (año, mes, día)
- ❖ TO_DATE()/TO_CHAR() → Convertir entre texto y fecha

Diccionario de Valores:

- ✓ year (año)
- ✓ quarter (trimestre)
- ✓ month (mes)
- ✓ day (día)
- ✓ hour (hora)
- ✓ minute (minuto)
- ✓ second (segundo)
- ✓ millisecond (milisegundo), entre otros.

FUNCIONES IDENTITY

- ❖ @@IDENTITY → devuelve el último IDENTITY generado en la sesión
- ❖ IDENT_CURRENT('Tabla') → devuelve el último IDENTITY generado en una tabla específica,
- ❖ SCOPE_IDENTITY() → el más seguro para obtener el último IDENTITY generado en tu misma sesión

INDICE DE COMANDOS DCL

COMANDOS BASICOS

- ❖ GRANT → Otorga permisos o privilegios a usuarios o roles sobre objetos de la base de datos.
- ❖ REVOKE → Quita permisos o privilegios previamente otorgados a usuarios o roles.

COMANDOS TCL

- ❖ COMMIT → Confirma los cambios realizados en la transacción actual.
- ❖ ROLLBACK → Revierte los cambios realizados en la transacción actual.
- ❖ SAVEPOINT → Define un punto dentro de una transacción al que se puede volver con ROLLBACK.
- ❖ SET TRANSACTION → Configura propiedades de la transacción (aislamiento, modo, etc.)

CONTROL DE SESIÓN Y CONEXIÓN

- ❖ SET → Configura variables de sesión, parámetros o opciones de la base de datos.
- ❖ USE → Selecciona la base de datos activa en la sesión.
- ❖ ALTER SESSION (Oracle) → Cambia parámetros de la sesión actual.

DEFINICIÓN AVANZADA DE OBJETOS

- ❖ CREATE SEQUENCE → Crea una secuencia para generar valores únicos automáticamente.
- ❖ CREATE SYNONYM → Crea un alias para un objeto de la base de datos (Oracle).
- ❖ CREATE MATERIALIZED VIEW → Crea una vista materializada que almacena datos físicamente (Oracle, PostgreSQL).
- ❖ ALTER SEQUENCE → Modifica propiedades de una secuencia existente.
- ❖ DROP SEQUENCE / SYNONYM / MATERIALIZED VIEW → Elimina estos objetos.

GESTIÓN DE USUARIOS Y ROLES

- ❖ CREATE USER / ALTER USER / DROP USER → Crear, modificar o eliminar usuarios.
- ❖ CREATE ROLE / ALTER ROLE / DROP ROLE → Gestionar roles de permisos.
- ❖ SET PASSWORD → Cambiar contraseña de un usuario.
- ❖ GRANT ROLE / REVOKE ROLE → Asignar o quitar roles a usuarios.

CONTROL AVANZADO DE OBJETOS

- ❖ ANALYZE / DBCC → Comandos de diagnóstico o mantenimiento (DBCC en SQL Server).
- ❖ EXPLAIN / EXPLAIN PLAN → Muestra el plan de ejecución de una consulta (optimización).
- ❖ LOCK TABLE / SELECT ... FOR UPDATE → Bloquea filas o tablas para control de concurrencia.

MANTENIMIENTO Y ADMINISTRACIÓN

- ❖ BACKUP / RESTORE → Respaldo y recuperación de la base de datos.
- ❖ CHECKPOINT → Fuerza la escritura de datos pendientes en disco (SQL Server, PostgreSQL).
- ❖ SHUTDOWN / STARTUP → Encender o apagar instancias de la base de datos (Oracle).
- ❖ TRUNCATE / PURGE → Eliminación rápida de datos, liberando espacio.

PERMISOS Y ROLES CON GRANT

PERMISOS SOBRE TABLAS

- ❖ SELECT → Permite leer datos de la tabla.
- ❖ INSERT → Permite agregar nuevos registros a la tabla.
- ❖ UPDATE → Permite modificar datos existentes en la tabla.
- ❖ DELETE → Permite eliminar registros de la tabla.
- ❖ REFERENCES → Permite crear claves foráneas que apunten a la tabla.
- ❖ TRIGGER → Permite crear triggers sobre la tabla

PERMISOS SOBRE VISTAS

- ❖ SELECT → Permite consultar datos de la vista.
- ❖ INSERT → Permite insertar datos a través de la vista (si es actualizable).
- ❖ UPDATE → Permite actualizar datos a través de la vista.
- ❖ DELETE → Permite eliminar datos a través de la vista.

PERMISOS SOBRE BD

- ❖ CREATE → Permite crear tablas, vistas u otros objetos dentro de la base de datos o esquema.
- ❖ ALTER → Permite modificar objetos existentes dentro de la base de datos o esquema.
- ❖ DROP → Permite eliminar objetos dentro de la base de datos o esquema.
- ❖ BACKUP/RESTORE → Permite hacer respaldo o restaurar la base de datos (SQL Server)

PERMISOS SOBRE PROCEDIMIENTOS

- ❖ EXECUTE → Permite ejecutar procedimientos almacenados o funciones.
- ❖ ALTER → Permite modificar la definición del objeto.
- ❖ DROP → Permite eliminar el objeto.
- ❖ PROCEDURE → Crear procedimientos almacenados

PERMISOS DE ADMINISTRACIÓN

- ❖ GRANT OPTION → Permite al usuario otorgar a otros usuarios los permisos que él posee.
- ❖ ALL PRIVILEGES → otorga todos los permisos posibles sobre el objeto (tabla, base de datos, etc.)
- ❖ CONTROL → para asignar permisos de transacciones

COMADOS CURSORES

GENERAL

- ❖ DECLARE **name_c** **OPTION** CURSOR FOR → Define un cursor con su consulta.
- ❖ OPEN CURSOR → Abre el cursor para lectura.
- ❖ FETCH → Recupera la siguiente fila del cursor.
- ❖ CLOSE CURSOR → Cierra el cursor.
- ❖ OPEN → Abre el cursor y ejecuta la consulta definida en DECLARE
- ❖ DECLARE **@variable1 TIPO**, **@variable2 TIPO**; → declara variables
- ❖ SET **@variable** = "valor" → Asignar valores obtenidos de filas del cursor a variables.
- ❖ FETCH NEXT FROM **name_c** INTO **@variable1**, **@variable2**;
- ❖ **@@FETCH_STATUS** → indica si el FETCH tuvo éxito
- ❖ CLOSE → Cierra el cursor
- ❖ DEALLOCATE → Libera completamente el cursor de memoria

FETCH

- FETCH FIRST → Primera fila (alternativa).
- FETCH PRIOR → Fila anterior.
- FETCH LAST → Última fila.
- FETCH ABSOLUTE n → Fila n-ésima.
- FETCH NEXT → siguiente fila

FETCH STATUS

- 0 → Se recuperó la fila correctamente.
- -1 → No hay más filas.
- -2 → Fila eliminada.

CONTROL DE FLUJO ASOCIADO A CURSORES

- WHILE ... END / LOOP → Permite iterar sobre las filas mientras se cumpla una condición.
- IF ... ELSE → Permite tomar decisiones basadas en los valores de las filas.
- BREAK / LEAVE → Salir de un bucle o ciclo antes de que termine (según motor).
- CONTINUE → Saltar a la siguiente iteración del bucle.
- WHILE → Itera mientras haya filas.

OPCIONES DE CURSOR

- STATIC → No refleja cambios posteriores
- DYNAMIC → refleja cambios en la tabla (inserciones, actualizaciones, eliminaciones)
- KEYSET → los cambios en los valores no claves se reflejan, pero filas nuevas no aparecen.
- FAST_FORWARD → permite recorrido hacia adelante, no permite actualizar ni retroceder
- FORWARD_ONLY → Similar a FAST_FORWARD, solo permite avanzar fila por fila.
- READ_ONLY → No permite actualizar los datos a través del cursor.
- SCROLL Permite moverse hacia adelante y atrás (por ejemplo FETCH PRIOR, FETCH ABSOLUTE)
- INSENSITIVE → Los cambios en la tabla no afectan el cursor una vez abierto.

OPCIONES ADICIONALES EN ORACLE Y POSTGRESQL

- FOR UPDATE → Bloquea las filas seleccionadas para que otros no las modifiquen mientras se recorre
- WITH HOLD (PostgreSQL) → Cursor permanece abierto después de un COMMIT.
- WITHOUT HOLD → Cursor se cierra automáticamente al hacer COMMIT.

TRANSACCIONES Y CURSORES

- COMMIT → Confirmar cambios realizados dentro de un cursor si se usan operaciones DML.
- ROLLBACK → Deshacer cambios dentro de un cursor en caso de error.

SECUENCIAS

GENERAL

- ❖ CREATE SEQUENCE → Crear una secuencia.
- ❖ ALTER SEQUENCE → Modificar parámetros de la secuencia
- ❖ DROP SEQUENCE → elimina una secuencia
- ❖ START WITH n → El primer número que generará la secuencia será “n”
- ❖ INCREMENT BY n → Cada vez que se use la secuencia, se incrementa en “n”
- ❖ NOCACHE → No almacena números en memoria
- ❖ CACHE → Permite almacenar en memoria varios valores de la secuencia para optimizar el rendimiento.
- ❖ START WITH → Define el valor inicial de la secuencia.
- ❖ MINVALUE → Establece el valor mínimo que puede generar la secuencia.
- ❖ MAXVALUE → Establece el valor máximo que puede generar la secuencia.
- ❖ CYCLE → Permite que la secuencia vuelva al valor inicial cuando alcanza el máximo (si aplica).
- ❖ NOCYCLE → Evita que la secuencia se reinicie al alcanzar el máximo.

COMANDOS PARA OBTENER VALORES

- ❖ NEXTVAL → Devuelve el siguiente valor de la secuencia y lo incrementa según el valor definido.
- ❖ CURRVAL → Devuelve el último valor generado de la secuencia en la sesión actual
- ❖ SETVAL → Permite establecer manualmente un valor actual de la secuencia (reiniciarla o ajustarla).

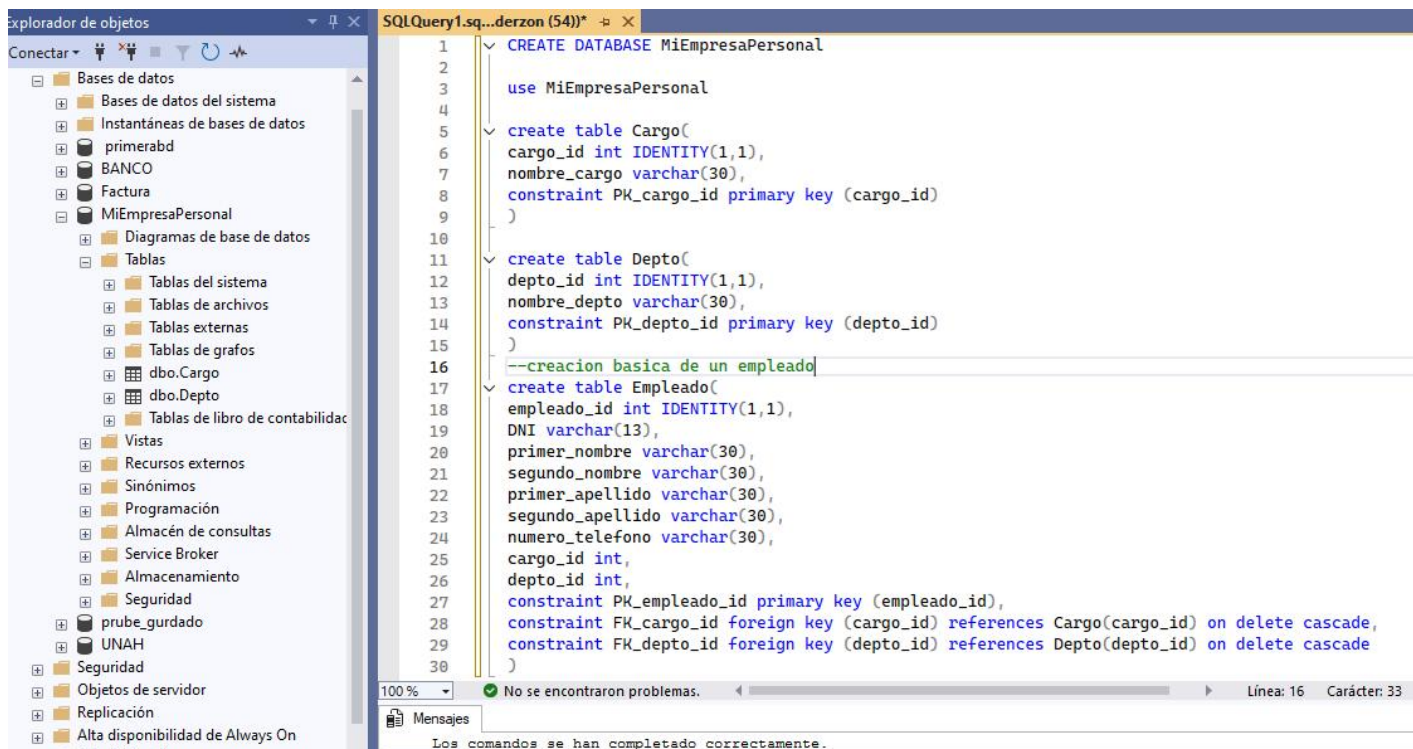
OPERADORES Y RESTRICCIONES ASOCIADOS A SECUENCIAS

- ❖ PRIMARY KEY → Usar la secuencia para generar valores únicos para una clave primaria.
- ❖ DEFAULT → Asignar automáticamente el siguiente valor de la secuencia a una columna
- REFERENCES → se puede usar en combinación con claves foráneas si la secuencia genera identificadores que se relacionan

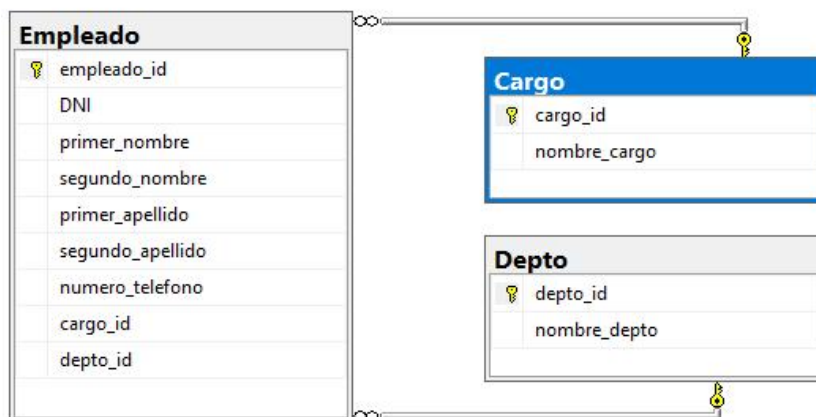
EJEMPLOS DEL USO DE DDL

CREATE

- ❖ **CREATE TABLE** nombre_tabla --sintaxis
- ❖ **CREATE DATABASES** nombre_BD --sintaxis



```
1 CREATE DATABASE MiEmpresaPersonal
2
3 use MiEmpresaPersonal
4
5 create table Cargo(
6 cargo_id int IDENTITY(1,1),
7 nombre_cargo varchar(30),
8 constraint PK_cargo_id primary key (cargo_id)
9 )
10
11 create table Depto(
12 depto_id int IDENTITY(1,1),
13 nombre_depto varchar(30),
14 constraint PK_depto_id primary key (depto_id)
15 )
16 --creacion basica de un empleado
17 create table Empleado(
18 empleado_id int IDENTITY(1,1),
19 DNI varchar(13),
20 primer_nombre varchar(30),
21 segundo_nombre varchar(30),
22 primer_apellido varchar(30),
23 segundo_apellido varchar(30),
24 numero_telefono varchar(30),
25 cargo_id int,
26 depto_id int,
27 constraint PK_empleado_id primary key (empleado_id),
28 constraint FK_cargo_id foreign key (cargo_id) references Cargo(cargo_id) on delete cascade,
29 constraint FK_depto_id foreign key (depto_id) references Depto(depto_id) on delete cascade
30 )
```



Descripción:

Creamos una BD (MiEmpresaPersonal) básica con 3 tablas (Cargo, Depto, Empleado) agregamos restricciones para sus llaves PK y FK para crear la relación entre las tablas como se ve en el diagrama, usamos IDENTITY (1,1) para que la PK se autoincremente en cada registro nuevo desde 1 y va de 1 en 1 y agregamos un ON DELETE CASCADE por si se elimina un registro que también lo hagan sus referencias

Comandos usados (CREATE, USE, IDENTITY, CONSTRAINT, REFERENCES, PK, FK, ON DELETE CASCADE)

```

31 |
32 | ✓ create table Cliente(
33 |     cliente_id int IDENTITY(1,1),
34 |     DNI varchar(13),
35 |     primer_nombre varchar(30),
36 |     segundo_nombre varchar(30),
37 |     primer_apellido varchar(30),
38 |     segundo_apellido varchar(30),
39 |     numero_telefono varchar(30),
40 |     cuenta_id int,
41 |     fecha_de_nacimiento datetime default getdate(),
42 |     constraint PK_cliente_id primary key (cliente_id),
43 | )
44 |
45 | ✓ create table Cuenta(
46 |     cuenta_id int IDENTITY(1,1),
47 |     numero_cuenta varchar (10) unique,
48 |     cliente_id int,
49 |     tipo_cuenta varchar (12) check (tipo_cuenta in ('Ahorros', 'Corriente', 'Plazo Fijo')),
50 |     saldo DECIMAL(15,2) DEFAULT 0.00,
51 |     fecha_apertura DATE,
52 |     estado varchar(12) check(estado in('Activa', 'Inactiva', 'Bloqueada')),
53 |     moneda VARCHAR(3) DEFAULT 'USD',
54 |     constraint PK_cuenta_id primary key (cuenta_id),
55 |     constraint FK_cliente_id foreign key (cliente_id) references Cliente(cliente_id) on delete cascade
56 | )
57 |

```

Descripción:

Usamos comando como **UNIQUE** esta es una restriccion que no permite que el valor creado se repita en otro registro, Tambien usamos **GETDATE()** para obtener la fecha del registro, en la tabla "Cuenta" usamos un **CHECK** que verifica que el campo se llene con un valor ya definido y agregamos **IN** para decir que puede ser cualquiera de los valores que le siguen (), usamos **DEFAULT** para asignar un valor automatico ya predefiniendo

DROP

BORRAS TABLAS

❖ `DROP TABLE nombre_tabla --sintaxis`

➤ **Caso:** Tenemos 2 tablas (Cargos y Empleados), la tabla Empleados depende de una FK que proviene de la tabla Cargo entonces, borrar la tabla Cargos

`drop table Cargos -- esto no eliminara la tabla!! ya que esa tabla tiene una PK de la cual depende la tabla empleados entonces habria que borrar Empleados primero para luego borrar Cargos)`

`--Si no quieres borrar toda la tabla dependiente, elimina solo la restriccion FK de la tabla dependiente`

`ALTER TABLE Empleados
drop constraint Nombre_FK;`

`--Después ya puedes borrar la tabla Cargo ya que no esta la restriccion de FK lo que le impedia que se borrara, la tabla Empleados conserva los datos
drop table Cargo;`

`--Definir la FK con ON DELETE CASCADE --borra datos de forma automática al eliminar registros de la tabla principal (al crear la tabla o modificando el CONSTRAINT con ALTER)`

Y lo mismo para objetos que estan relacionados a la tabla

BORRAR OBJETOS

❖ `DROP Tipo_objeto NombreObjeto --sintaxis`

Tipos_objetos:

✧ Vista

`DROP VIEW NombreVista;`
--Si la Vw depende o es dependecia de otra Vw Quitar dependencias

✧ Procedimiento almacenado

`DROP PROCEDURE NombreSP;`
--Si tiene o es dependecia de otra Vw Quitar dependencias

✧ Función

`DROP FUNCTION NombreFuncion;`
--Si la f depende o es dependecia de otra f Quitar dependencias

✧ Trigger

`DROP TRIGGER NombreTrigger;`
-- Si un trigger está activando otro trigger Deshabilitarlo primero:

`DISABLE TRIGGER MiTrigger ON MiTabla;`
`DROP TRIGGER MiTrigger;`

✧ Índices

`DROP INDEX NombreIndice ON MiTabla;`
--Si es PK o Unique, Constraint, quitar la restricción.
`ALTER TABLE MiTabla DROP CONSTRAINT PK_MiTabla;`
-- o para índices simples
`DROP INDEX MiIndice ON MiTabla;`

✧ Roles y usuarios

`DROP ROLE MiRol;`
--No se puede borrar si tiene miembros asignados, solucion: sacarlos

`DROP USER MiUsuario;`
--No se puede borrar un usuario si tiene objetos propios en la BD

✧ Synonyms

`DROP SYNONYM MiSinonimo`

✧ Schemas

`DROP SCHEMA MiEsquema`
--No se puede borrar si contiene obj dentro, mover o borrarlos todos

--Foreign Key / Primary Key / Unique / Check / Default
`ALTER TABLE MiTabla DROP CONSTRAINT NombreConstraint`

BORRAR BDs

❖ `DROP DATABASE nombre_BD --sintaxis`

➤ **Caso:** Tenemos una BD que tiene usuarios conectados u objetos en uso

```
DROP DATABASE Mi_BD;
```

```
ALTER DATABASE Mi_BD SET SINGLE_USER WITH ROLLBACK IMMEDIATE;  
DROP DATABASE Mi_BD; -- si usamos DROP con usuarios conectados dara error  
usamos este comando para evitar eso
```

```
ALTER DATABASE Mi_BD  
--cambiamos la configuración de la BD
```

```
SET SINGLE_USER (#)  
-- Cierra el acceso a todos los demás usuarios, te reserva la BD para ti  
solo
```

```
WITH ROLLBACK IMMEDIATE (#)  
-- Desconecta a todos los usuarios de inmediato, cancelando sus  
transacciones abiertas, (si no pones esto, te obliga a esperar a que  
cierren sesión).
```

```
DROP DATABASE Mi_BD;-- Ahora si puedes eliminar la BD sin error
```

➤ **Caso:** Jobs de SQL Agent, Tienes que editar o eliminar el Job en SQL Server Agent antes de hacer DROP DATABASE.

```
EXEC msdb.dbo.sp_delete_job @job_name = 'NombreDelJob';
```

➤ **Caso:** Linked Servers, Si la BD está referenciada en un Linked Server, debes eliminar o modificar el Linked Server.

```
EXEC sp_dropserver 'NombreLinkedServer', 'droplogins';
```

➤ **Caso:** Replication Si la BD es Publisher, Distributor o Subscriber, primero debes deshabilitar la replicación

```
EXEC sp_removedbreplication @dbname = 'Mi_BD';
```

➤ **Caso:** Mirroring Si está en Database Mirroring, debes quitar el mirroring:

```
ALTER DATABASE MiBD SET PARTNER OFF;
```

➤ **Caso:** Availability Groups (AlwaysOn) Si la BD está en un AG, primero quítala del grupo:

```
ALTER AVAILABILITY GROUP MiAG REMOVE DATABASE MiBD;
```


ALTER

SINTAXIS

```
ALTER TABLE name_table --modifica una tabla
ALTER COLUMN name_column -- modifica una columna
ADD constraint -- modifica un constraint
ADD campo tipo y condiciones -- agrega un campo y su tipo
```

EJEMPLOS

Ejemplo_1:

Se creo la tabla empleados se desea agregar un constraint a la primary key:

```
create table Empleado (
empleado_id nvarchar (5),
nombre_empleado nvarchar (100),
sueldo_empleado money
)
-- el constraint no acepta campos donde pueda caer un null
ALTER TABLE Empleado --le agregamos no null con alter colum
ALTER COLUMN empleado_id NVARCHAR(5) NOT NULL ;

-- agregamos el constraint con ADD
alter table Empleado add constraint empleadoPK primary Key (empleado_id)
-- alter table edita la tabla
ALTER TABLE empleado
ADD empleado_id INT IDENTITY(1,1) UNIQUE;
```

EJEMPLOS DEL USO DE DML

SELECT

sintaxis básica de una consulta select:

```
select campos from name_table
go
-- go = final de un lote de consulta
-- from = tabla de procedencia
```

Ejemplo:

```
select * from Factura
go
-- * = todo
```

sintaxis de un select mostrando campos seleccionados:

```
select name_campo1,name_campo2,name_campon_n from name_table k
go
```

-- k es una alias para la tabla, puede ser cualquier otra letra o palabra
-- es como un objeto y se accede a ella con un punto '.'

Ejemplo:

cualquiera de las 2 formas dara el mismo resultado

```
-- de forma directa
select FacturaId, total_pagar from Factura f
go
-- usando el alias f.
select f.FacturaId, f.total_pagar from Factura f
go
```

ORDER BY



```
SELECT *  
FROM Clientes  
ORDER BY ID_cliente  
OFFSET 5 ROWS FETCH NEXT 10 ROWS ONLY;
```

	ID_cliente	nombre	direccion	telefono	email	ciudad
1	7	Laura Fernández	Calle Luna 99	555-4321	laura@example.com	Lima
2	8	Sofía Ruiz	Avenida Libertad 45	555-1122	sofia@example.com	Caracas
3	9	Diego Jiménez	Calle Sol 33	555-3344	diego@example.com	Cancún
4	10	Cristina Gómez	Camino Viejo 77	555-5566	cristina@example.com	Bogotá
5	11	Roberto Díaz	Calle Mar 22	555-7788	roberto@example.com	Ciudad de México

“OFFSET 5 ROWS FETCH NEXT 10 ROWS ONLY” no mostrara los registros apartir de la fila 6 y nos traera los siguientes 10

FUNCIONES DE AGREGACION

UNION

La cláusula UNION se usa para combinar los resultados de dos o más consultas

Sintaxis **básica:**

```
SELECT columna1, columna2 FROM tabla1  
UNION  
SELECT columna1, columna2 FROM tabla2;
```

-- Para incluir duplicados

Sintaxis **básica:**

```
SELECT columna1, columna2 FROM tabla1  
UNION ALL  
SELECT columna1, columna2 FROM tabla2;
```

CTE (COMMON TABLE EXPRESSION)

SINTAXIS

Los CTE es una tabla temporal que existe solo durante la ejecución de una consulta SQL.

- Divide consultas complejas en partes
- Más fácil de entender y modificar
- Permite consultas recursivas aplicables en árboles, jerarquías
- Es reutilizable

```
WITH name_cte AS (  
    -- aqui aplicamos las consulta que queremos ver  
    SELECT columns FROM name_table  
    WHERE conditions  
)  
SELECT * FROM nombre_cte; -- aqui usamos el CTE de forma inmediata
```

EJEMLOS

#1

-- CTE simple

```
WITH EmpleadosVentas AS ( SELECT nombre, salario, departamento FROM empleados  
    WHERE departamento = 'Ventas'  
)  
SELECT * FROM EmpleadosVentas;
```

#2

-- CTE con cálculos

```
WITH SalariosPromedio AS (  
    SELECT departamento, AVG(salario) as salario_promedio  
    FROM empleados GROUP BY departamento  
)  
SELECT e.nombre, e.salario, e.departamento, sp.salario_promedio  
FROM empleados e  
INNER JOIN SalariosPromedio sp ON e.departamento = sp.departamento  
WHERE e.salario > sp.salario_promedio;
```

SUBCONSULTAS

SINTAXIS

-- una sub consulta es una consulta dentro de otra consulta
-- agg 'as total' para asignar un alias al campo creado por la subconsulta

```
select t.campo_1, t.campo_2, t.campo_3, t.campo_n
(select campos|funcion from name_table) as alias -- esta es una subconsulta
from name_Table t
go
```

SUBCONSULTA DE UNA FILA

```
select * from Cliente
where NOMBRE in (select clienteID from cliente where id_CLIENTE= 'PR01')
--busca el nombre de un cliente en de la tabla clientes cuya ID es PR01
```

SUBCONSULTA MÚLTIPLES FILAS

```
select * from Cliente c
where c.clienteID in (select c.clienteID from cliente c where c.P_nombre like
'%a%')
-- esta consulta me busca todo de la tabla clientes cuya ID son de aquellos
usuarios que tienen una A en su nombre
```

SUBCONSULTAS CORRELACIONADAS

-- Referencian columnas de la consulta externa.

```
SELECT e1.nombre, e1.salario, e1.departamento_id
FROM empleados e1
WHERE salario > ( SELECT AVG(e2.salario) FROM empleados e2
WHERE e2.departamento_id = e1.departamento_id)
```

SUBCONSULTAS CORRELACIONADAS

```
SELECT d.nombre_depto, alias_emple.promedio FROM deptos d
INNER JOIN (SELECT depto_id, AVG(salario) as promedio, COUNT(*) as total_emple
FROM empleados d GROUP BY depto_id
) alias_emple ON d.depto_id = alias_emple.depto_id;
-- 'sub.campo' para usar un campo de la subconsulta como vemos en
-- alias_emple.depto_id y en alias_emple.promedio donde "depto_id" y "promedio"
-- son campos de las subconsulta
```

SUBCONSULTAS CON INSERT

```
INSERT INTO empleados (nombre, salario, fecha_ingreso)
--subconsulta
SELECT nombre, salario, GETDATE()
FROM empleados
WHERE fecha_contratacion < '2020-01-01';
--insertar datos de otra tabla usando subconsultata
```

CCON EXIST | NO EXIST | ANY | SOME | ALL

-- mostrar los clientes que han hecho al menos una orden

```
SELECT cl.P_nombre , clienteID, 1 as n_compras
-- imprimir un campo con unos por cada registro
FROM cliente cl -- tabla sobre la que vamos a trabajar
WHERE EXISTS (SELECT 1 FROM dbo.ORDEN_SERVICIO fa WHERE fa.clienteID =
Cl.ClienteID)-- 'exists' se asegura de que existe al menos una fila relacionada
```

SUBCONSULTAS CON UPDATE

```
UPDATE empleados
SET salario = salario * 1.10
WHERE departamento_id IN (SELECT departamento_id FROM departamentos WHERE
ubicacion = 'CORTES')
-- obtiene el ID del depto CORTES y lo usa como un escalar para validar el where
```

SUBCONSULTAS CON CTE

```
WITH EmpleadosDepartamento AS (
    SELECT departamento_id AVG(salario) as salario_promedio FROM empleados
    GROUP BY departamento_id
)
SELECT e.nombre, e.salario, ed.salario_promedio
FROM empleados e
INNER JOIN EmpleadosDepartamento ed ON e.departamento_id = ed.departamento_id
WHERE e.salario > ed.salario_promedio;
```

JOINS

INNER JOIN

Devuelve registros que coinciden en ambas tablas, La relación se define en el **ON**
SINTAXIS:

```
SELECT columns
FROM name_table_1 t1
INNER JOIN name_table2 t2 ON t1.column = t2.column;
```

MULTIPLES JOINS

```
SELECT
    c.nombre as cliente, p.monto, pr.nombre as producto
FROM clientes c
INNER JOIN pedidos p ON c.id = p.cliente_id
INNER JOIN productos pr ON p.producto_id = pr.id;
-- conecta la tabla clientes con pedidos y pedidos con productos donde se cumplan
las condiciones de = donde los ID sean los mismos
```

INNER JOIN CON CONDICIONES

```
SELECT c.nombre, p.monto
FROM clientes c
INNER JOIN pedidos p ON c.id = p.cliente_id
WHERE p.monto > 1000;
```

LEFT JOIN

Todos los registros de la tabla izquierda mas coincidencias de la derecha

```
SELECT columnas
FROM tabla1
LEFT JOIN tabla2 ON tabla1.columna = tabla2.columna;
```

```
-- Ejemplo
SELECT c.nombre, p.monto
FROM clientes c
LEFT JOIN pedidos p ON c.id = p.cliente_id;
```

RIGHT JOIN

Todos los registros de la tabla derecha mas coincidencias de la izquierda

```
-- Ejemplo
SELECT columnas
FROM tabla1
RIGHT JOIN tabla2 ON tabla1.columna = tabla2.columna;

SELECT c.nombre, p.monto
FROM clientes c
RIGHT JOIN pedidos p ON c.id = p.cliente_id;
```


CROSS JOIN

Producto cartesiano de todas las combinaciones posibles

```
SELECT columnas
FROM tabla1
CROSS JOIN tabla2;
```

-- Ejemplo

```
SELECT c.nombre, p.monto
FROM clientes c
CROSS JOIN pedidos p;
```

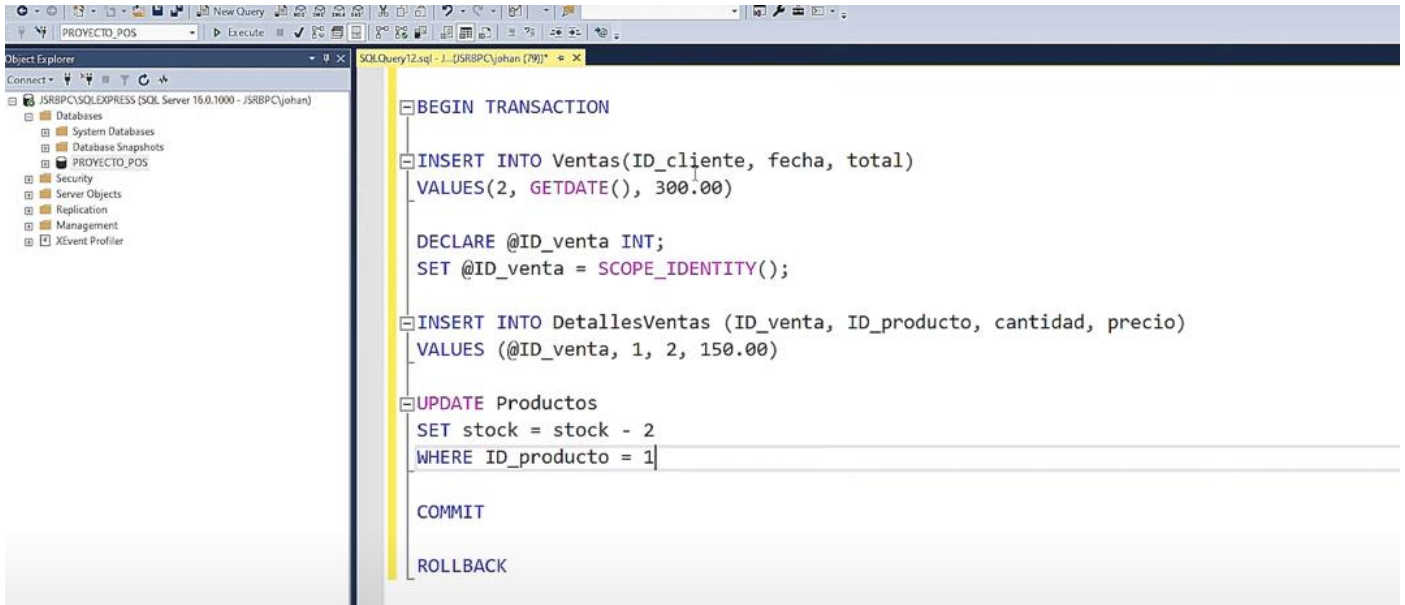
COMBINADOS

-- Consulta con INNER JOIN y LEFT JOIN

```
SELECT
    d.nombre as departamento,
    e.nombre as empleado,
    p.nombre as proyecto
FROM departamentos d
INNER JOIN empleados e ON d.id = e.depto_id
LEFT JOIN proyectos p ON e.id = p.empleado_id;
```

EJEMPLOS DEL USO DE TCL

COMIT AND ROLLBACK

A screenshot of the Microsoft SQL Server Enterprise Manager interface. The left pane shows the 'Object Explorer' with a tree view of the 'PROYECTO_POS' database, including 'Databases', 'System Databases', 'Database Snapshots', 'Security', 'Server Objects', 'Replication', 'Management', and 'XEvent Profiler'. The right pane shows a 'SQL Query' window with a script for a transaction. The script includes: BEGIN TRANSACTION, INSERT INTO Ventas (ID_cliente, fecha, total) VALUES (2, GETDATE(), 300.00), DECLARE @ID_venta INT; SET @ID_venta = SCOPE_IDENTITY();, INSERT INTO DetallesVentas (ID_venta, ID_producto, cantidad, precio) VALUES (@ID_venta, 1, 2, 150.00), UPDATE Productos SET stock = stock - 2 WHERE ID_producto = 1, COMMIT, and ROLLBACK.

```
BEGIN TRANSACTION

INSERT INTO Ventas(ID_cliente, fecha, total)
VALUES(2, GETDATE(), 300.00)

DECLARE @ID_venta INT;
SET @ID_venta = SCOPE_IDENTITY();

INSERT INTO DetallesVentas (ID_venta, ID_producto, cantidad, precio)
VALUES (@ID_venta, 1, 2, 150.00)

UPDATE Productos
SET stock = stock - 2
WHERE ID_producto = 1

COMMIT

ROLLBACK
```

Sintaxis de una transacción, en este caso hacemos una transacción donde modificamos tablas así como agregamos nuevos registros, primero agregamos un registro a la tabla ventas, declaramos una variable a la cual le vamos a asignar el último valor insertado a una identidad usando **SCOPE_IDENTITY()**, ingresamos los valores a la tabla DetallesVenta y con update modificamos la tabla productos restando lo que se vendió a la tabla DetallesVentas.

Finalmente ejecutamos **COMIT** si queremos guardar la transacción o **ROLLBACK** si queremos deshacerla.

ÍNDICES

Son estructuras que mejoran la velocidad de las operaciones de consulta en tablas, como el índice de un libro que te permite encontrar información rápido sin leer todo el libro

SINTAXIS:

```
CREATE INDEX IX_name_index ON table(column)
```

TIPOS

-- imagina que tenemos esta tabla, la usaremos para definir los tipos de índices

```
CREATE TABLE empleados_1 (  
    id INT PRIMARY KEY,  
    nombre VARCHAR(100),  
    apellido VARCHAR(100),  
    email VARCHAR(150),  
    departamento_id INT,  
    salario DECIMAL(10,2),  
    fecha_contratacion DATE,  
    ciudad VARCHAR(50)  
)
```

Índice Non-Clustered Básico

-- Para búsquedas por apellido

```
CREATE INDEX IX_empleados_apellido ON empleados(apellido)
```

Índice Compuesto

-- Para búsquedas por departamento y salario

```
CREATE INDEX IX_empleados_depto_salario ON empleados(departamento_id, salario);
```

Índice Único

-- Para email único

```
CREATE UNIQUE INDEX IX_empleados_email ON empleados(email);
```

Índice con INCLUDE

-- Incluye columnas adicionales para evitar key lookups

```
CREATE INDEX IX_empleados_depto_include ON empleados(departamento_id)  
INCLUDE (nombre, apellido, salario);
```

Índice Filtrado WHERE

-- Solo para empleados activos (si tuviéramos columna estado)

```
CREATE INDEX IX_empleados_activos ON empleados(departamento_id)  
WHERE estado = 'Activo';
```

Índice para ORDER BY

-- Optimiza ordenamientos por fecha

```
CREATE INDEX IX_empleados_fecha_contratacion ON empleados(fecha_contratacion  
DESC);
```

USO DE ÍNDICES

Usa índice simple:

```
SELECT * FROM empleados WHERE apellido = 'García';
```

Usa índice compuesto:

```
SELECT nombre, salario  
FROM empleados  
WHERE departamento_id = 5 AND salario > 50000;
```

Usa índice con INCLUDE:

```
SELECT nombre, apellido, salario  
FROM empleados  
WHERE departamento_id = 3;
```

VER ÍNDICES EXISTENTES:

```
-- Ver todos los índices de la tabla  
EXEC sp_helpindex 'empleados';
```

PARTICIONAMIENTO DE TABLAS

Dividir una tabla grande en fragmentos más pequeños 'particiones' basados en un criterio

CUANDO SÍ particionar:

- Tablas muy grandes, millones de registros
- Patrones de acceso por rangos por ejemplo por fecha, región
- Necesidad de mantener/archivar datos antiguos
- Mejorar performance de consultas por rangos

CUANDO NO particionar:

- Tablas pequeñas/medianas
- Sin patrón claro de acceso
- Recursos limitados

SINTAXIS

Crear Función de Partición

```
CREATE PARTITION FUNCTION nombre_funcion (tipo_dato)
AS RANGE RIGHT FOR VALUES (valor1, valor2, valor3);
```

Crear Esquema de Partición

```
CREATE PARTITION SCHEME nombre_esquema
AS PARTITION nombre_funcion
TO (fg1, fg2, fg3, fg4);
-- donde fg1 son filegroup
```

Crear Tabla Particionada

```
CREATE TABLE nombre_tabla (
    columnas...
) ON nombre_esquema(columna_particion);
```

VISTAS

Son consultas guardadas que se comportan como tablas virtuales

SINTAXIS

VISTAS SIMPLES

```
CREATE VIEW name_view
AS
SELECT columns
FROM tables
WHERE conditions
```

USARLAS

```
SELECT * FROM name_view;
SELECT * FROM name_view WHERE column = 'condition'
```

VISTAS COMPLEJAS

```
CREATE VIEW name_view
AS
SELECT
    t1.columna1,
    t2.columna2,
    COUNT(t3.column3) as total,
    SUM(t4.column4) as sum
FROM table1 t1
INNER JOIN table2 t2 ON t1.id = t2.table1_id
LEFT JOIN table3 t3 ON t2.id = t3.table2_id
GROUP BY t1.column1, t2.column2
HAVING COUNT(t3.column3) > 5;
```

EJEMPLOS VW

#1

--imagina que tenemos esta tabla

```
CREATE TABLE empleados_1 (
    id INT,
    nombre VARCHAR(100),
    salario DECIMAL(10,2),
    departamento VARCHAR(50),
    email VARCHAR(100)
)
```

```
CREATE VIEW vw_empleados_publicos
AS
SELECT
    Nombre ,departamento,email
FROM empleados_1
WHERE departamento IN ('Ventas', 'Soporte', 'Marketing');
```

```
SELECT * FROM vw_empleados_publicos;
SELECT * FROM vw_empleados_publicos
WHERE departamento = 'Ventas';
```

VISTAS MATERIALIZADAS

Son vistas que almacenan físicamente los datos en disco, como una tabla

Vista normal: Consulta virtual se ejecuta al llamarla

Vista materializada: Datos pre-calculados y almacenados

IDEALES para:

- Consultas complejas con agregaciones
- Datos que cambian poco
- Reportes frecuentes que son lentos

EVITAR cuando:

- Datos cambian frecuentemente
- Consultas simples
- Poco uso de la vista

```
CREATE VIEW nombre_vista
WITH SCHEMABINDING
AS
consulta_sql;
CREATE UNIQUE CLUSTERED INDEX nome_index
ON nome_view (column);
```

Ejemplos

--imagina que tenemos estas tablas

```
CREATE TABLE ventas (
    id INT IDENTITY PRIMARY KEY,
    fecha DATE,
    producto_id INT,
    cantidad INT,
    monto DECIMAL(10,2)
)
```

```
CREATE TABLE productos (
    id INT PRIMARY KEY,
    nombre VARCHAR(100),
    categoria VARCHAR(50)
)
```

-- creamos la vwm

```
CREATE VIEW vw_ventas_totales
WITH SCHEMABINDING
AS
SELECT
    p.categoria,
    YEAR(v.fecha) as año, COUNT_BIG(*) as total_ventas,
    SUM(v.monto) as ingreso_total
FROM dbo.ventas v
INNER JOIN dbo.productos p ON v.producto_id = p.id
GROUP BY p.categoria, YEAR(v.fecha);
```

-- creamos su indice

```
CREATE UNIQUE CLUSTERED INDEX IX_vw_ventas_totales
ON vw_ventas_totales(categoria, año);
```

-- llamamos a la vwm

```
SELECT * FROM vw_ventas_totales
WHERE año = 2024;
```

GESTIÓN DE USUARIOS

CREAR USUARIOS

-- Crear un login en la BD, Un mismo login puede tener usuarios distintos en varias bases de datos del servidor (EL LOGIN SE CREA A NIVEL SERVIDOR)

```
create login name_login with password = 'password'
```

-- Crear un usuario en la BD

```
create user name_usuario for login name_login
```

-- Asignar permisos al rol a una BD

```
GRANT permisos ON database::name_BD TO name_user
```

-- Asignar permisos al rol a una tabla a un usuario

```
GRANT permisos ON esquema.name_table TO name_user
```

-- Asignar permisos todo un esquema a un usuario

```
grant permisos on schema::name_esquema to name_user
```

-- Quitar permiso a un usuario

```
REVOKE SELECT, UPDATE ON dbo.Empleados FROM usuario_reportes;
```

-- Dar control total del esquema a un usuario

```
GRANT CONTROL ON SCHEMA::nombre_esquema TO usuario_reportes;
```

CREAR ROLES

-- Crear un rol

```
CREATE ROLE name_role
```

-- Asignar permisos al rol a una BD

```
GRANT SELECT ON database::name_BD TO name_role
```

-- Asignar permisos al rol a una tabla a un usuario

```
GRANT SELECT ON esquema.name_table TO name_role
```

-- Asignar permisos todo un esquema a un rol

```
grant delete on schema::name_esquema to name_role
```

-- Agregar un usuario al rol

```
ALTER ROLE name_role ADD MEMBER name_user
```

--sacar a un usuario del rol

```
ALTER ROLE name_role DROP MEMBER name_user;
```

-- Quitar permiso a un rol

```
REVOKE INSERT ON dbo.AuditoriaCambiosSalario FROM name_role;
```

-- Dar control total del esquema al rol

```
GRANT CONTROL ON SCHEMA::nombre_esquema TO name_role;
```


AUDITORIA PARA ROLES Y USUARIOS

```
-- ver tablas de usuarios
select * from sys.sysusers

-- Ver roles existentes
SELECT * FROM sys.database_principals WHERE type = 'R';

-- Ver permisos de un rol específico
SELECT dp.permission_name, dp.state_desc, ob.name AS objeto
FROM sys.database_permissions dp
JOIN sys.objects ob ON dp.major_id = ob.object_id
WHERE dp.grantee_principal_id = USER_ID('rol_gerencia');

-- Ver roles asignados a cada usuario
SELECT dp1.name AS Usuario, dp2.name AS Rol
FROM sys.database_role_members drm
JOIN sys.database_principals dp1 ON drm.member_principal_id =
dp1.principal_id
JOIN sys.database_principals dp2 ON drm.role_principal_id =
dp2.principal_id;
-- Ver permisos directos de cada usuario
SELECT dp.class_desc, dp.permission_name, dp.state_desc, ob.name AS Objeto,
usr.name AS Usuario
FROM sys.database_permissions dp
JOIN sys.database_principals usr ON dp.grantee_principal_id =
usr.principal_id
LEFT JOIN sys.objects ob ON dp.major_id = ob.object_id
ORDER BY usr.name;
```

EJEMPLOS

Ejemplo1

Dar permisos de lectura sobre todas las tablas a un usuario

```
CREATE LOGIN usuario_reportes WITH PASSWORD = 'Reportes2025!';
CREATE USER usuario_reportes FOR LOGIN usuario_reportes;
GO
-- Declaramos variable para el script dinámico
DECLARE @sql NVARCHAR(MAX) = '';

-- Generamos GRANT SELECT para todas las tablas en dbo
SELECT @sql = @sql + 'GRANT SELECT ON dbo.' + QUOTENAME(name) + ' TO
usuario_reportes;' + CHAR(13)
FROM sys.tables
WHERE schema_id = SCHEMA_ID('dbo');

-- Ejecutamos el script
EXEC sp_executesql @sql;

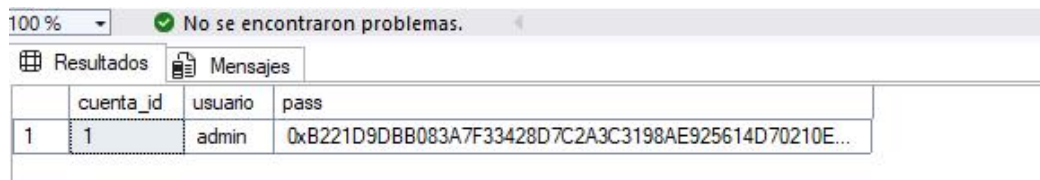
(Resumido)
-- asigna el permiso a toda la BD al usuario
GRANT SELECT ON database::FacturaCajero TO usuario_reportes;
```

ENCRYPTACIÓN DE DATOS

HAHS

```
create table cuentas (  
  cuenta_id int identity(1,1),  
  usuario varchar(20),  
  pass VARBINARY(64) NOT NULL, --el tipo de datos debe ser VARNINARY(n)  
  constraint PK_cuenta_id_1 primary key (cuenta_id)  
)  
  
-- Insertar usuario con contraseña hasheada  
INSERT INTO cuentas(usuario, pass)  
VALUES ('admin', HASHBYTES('SHA2_256', 'hola'));  
  
-- ver la tabla  
select * from cuentas
```

Esta es la sintaxis basica para sifrar datos en sql server con los **HASH**, se vera asi:



	cuenta_id	usuario	pass
1	1	admin	0xB221D9DBB083A7F33428D7C2A3C3198AE925614D70210E...

CREACION DE ESQUEMAS

```
--crear esquema
CREATE SCHEMA name_esquema;

-- Insertar en la tabla clientes del esquema ventas
INSERT INTO name_esquema.name_table (campo1, campo2)
VALUES ('value 1', 'value 2')

-- ver tabla clientes del esquema ventas
SELECT c.Nombre, c.ClienteID FROM ventas.Clientes c

--Esto crea el esquema rrhh y asigna como propietario al usuario/rol usr_Richard.
CREATE SCHEMA rrhh AUTHORIZATION usr_Richard;

--mover tablas entre esquemas
ALTER SCHEMA esquema_destino TRANSFER esquema_origen.nombre_tabla

--Eliminar un esquema
DROP SCHEMA rrhh;

--Permitir SELECT a todos los objetos del esquema ventas para el usuario
GRANT SELECT ON SCHEMA::ventas TO User1;

-- Permitir CRUD completo (DML)
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::ventas TO User1;

-- Quitar permisos de inserción, actualización y eliminación
REVOKE INSERT, UPDATE, DELETE ON SCHEMA::ventas FROM User1;

-- Bloquear SELECT aunque el usuario tenga permisos en otra parte
DENY SELECT ON SCHEMA::ventas TO User1;

-- Quitar permiso de SELECT a usuario sobre una tala que pertenece a un esquema
REVOKE SELECT ON ventas.Clientes FROM User1;

--Renombrar un objeto dentro de un esquema
EXEC sp_rename 'ventas.Clientes', 'Clientes_Nuevo';

SELECT * FROM sys.schemas; --ver tabla de esquemas existentes

--Usar sinónimos apuntando a objetos en otro esquema
CREATE SYNONYM ventas.OrdenesExternas FOR otroSchema.Ordenes;
```

EJEMPLOS

Ejemplo 1:

```
--crear esquema
CREATE SCHEMA ventas;

--Crear tabla Clientes en el esquema ventas
CREATE TABLE ventas.Clientes (
    ClienteID INT PRIMARY KEY IDENTITY(1,1),
    Nombre NVARCHAR(100) NOT NULL,
    Correo NVARCHAR(100) UNIQUE,
    FechaRegistro DATE DEFAULT GETDATE()
)

--insertar datos a la tabla del esquema ventas
INSERT INTO ventas.Clientes (Nombre, Correo)
VALUES ('value 1', 'value 2')
```

Ejemplo 2:

Mover todas la tablas de un esquema a otro de forma dinaminca

```
-- Script dinámico para mover todas las tablas de dbo a ventas
DECLARE @sql NVARCHAR(MAX) = ''

SELECT @sql = @sql + 'ALTER SCHEMA ventas TRANSFER dbo.' + QUOTENAME(name) + ';'
+ CHAR(13)
FROM sys.tables
WHERE schema_id = SCHEMA_ID('dbo')-- Solo tablas del esquema dbo

-- Ejecutamos el script generado
EXEC sp_executesql @sql
```

Ejemplo 3:

Mover ciertas tablas de un esquema a otro de forma dinaminca pero no todas (filtradas)

```
DECLARE @sql NVARCHAR(MAX) = ''

--Creamos una tabla temporal con los nombres de las tablas que queremos mover
DECLARE @TablasMover TABLE (NombreTabla NVARCHAR(128))

INSERT INTO @TablasMover (NombreTabla)
VALUES ('Tabla1'), ('Tabla3'), ('Tabla7');--tablas que quieras mover

-- Generamos el script dinámico
SELECT @sql = @sql + 'ALTER SCHEMA ventas TRANSFER dbo.' + QUOTENAME(NombreTabla)
+ ';' + CHAR(13)
FROM @TablasMover

-- Ejecutamos el script
EXEC sp_executesql @s
```

CURSORES

SINTAXIS:

```
DECLARE @variable_1 TIPO, @variable_2 TIPO;
```

```
DECLARE name_cursor CURSOR FOR  
    SELECT t.campo_1, t.campo_1  
    FROM name_table t;
```

```
OPEN name_cursor;  
FETCH NEXT FROM name_cursor INTO @variable_1, @variable_2;
```

```
WHILE @@FETCH_STATUS = 0  
BEGIN  
    Cuerpo de alguna función  
END;
```

```
CLOSE name_cursor;  
DEALLOCATE name_cursor;
```

EJEMPLO:

--Consultar

```
DECLARE @id INT, @nombre NVARCHAR(50);
```

```
DECLARE cursor_clientes CURSOR FOR  
    SELECT c.clienteID, c.P_nombre  
    FROM Cliente c;
```

```
OPEN cursor_clientes;  
FETCH NEXT FROM cursor_clientes INTO @id, @nombre;
```

```
WHILE @@FETCH_STATUS = 0  
BEGIN  
    PRINT 'Cliente: ' + CAST(@id AS NVARCHAR) + ' - ' + @nombre;  
    FETCH NEXT FROM cursor_clientes INTO @id, @nombre;  
END;
```

```
CLOSE cursor_clientes;  
DEALLOCATE cursor_clientes;
```

EXEC

BACKUPS

- Full Backup: Diariamente (noche)
- Differential Backup: Varias veces al día (cada 4-6 horas)
- Transaction Log Backup: Cada 15-30 minutos (en bancos puede ser cada 5 min)

FULL BACKUP (Respaldo completo)

-- Copia toda la base de datos en un archivo .bak

```
BACKUP DATABASE MiBase  
TO DISK = 'C:\Backups\MiBase_Full.bak'  
WITH FORMAT, INIT;
```

--WITH FORMAT → crea un nuevo medio de backup

--INIT → sobrescribe cualquier backup anterior en ese archivo

RESTAURARLA

```
RESTORE DATABASE MiBase  
FROM DISK = 'C:\Backups\MiBase_Full.bak'  
WITH REPLACE;
```

--WITH REPLACE → sobrescribe la base existente si ya existe

DIFFERENTIAL BACKUP (Respaldo diferencial)

```
BACKUP DATABASE MiBase  
TO DISK = 'C:\Backups\MiBase_Diff.bak'  
WITH DIFFERENTIAL;
```

--Solo copia los cambios que están en el log de transacciones.

--Permite restaurar la base de datos hasta un punto específico en el tiempo.

--“differential” → significa solo los cambios desde el último Full Backup

RESTAURARLA

--Se hace en dos pasos:

-- Primero restaurar Full Backup, pero no finalizamos

```
RESTORE DATABASE MiBase  
FROM DISK = 'C:\Backups\MiBase_Full.bak'  
WITH NORECOVERY;
```

-- Luego restaurar Differential Backup

```
RESTORE DATABASE MiBase  
FROM DISK = 'C:\Backups\MiBase_Diff.bak'  
WITH RECOVERY;
```

--NORECOVERY → mantiene la BD para poder aplicar más backups.

--RECOVERY → finaliza la restauración y deja la base lista para usar

TRANSACTION LOG BACKUP (Respaldo del log de transacciones)

```
BACKUP LOG MiBase  
TO DISK = 'C:\Backups\MiBase_Log.trn';
```

--BACKUP LOG → indica que estamos haciendo backup del log de transacciones

--.trn → extensión común para backups de logs (puede ser .bak también)

RESTAURARLA

--Necesitas al menos un Full Backup antes, y opcionalmente un Differential.

-- Restaurar Full Backup (siempre)

```
RESTORE DATABASE MiBase
FROM DISK = 'C:\Backups\MiBase_Full.bak'
WITH NORECOVERY; --aplicar mas backups
```

-- Restaurar Differential Backup (si se necesita)

```
RESTORE DATABASE MiBase
FROM DISK = 'C:\Backups\MiBase_Diff.bak'
WITH NORECOVERY; --aplicar mas backups
```

-- Restaurar Log Backup

```
RESTORE LOG MiBase
FROM DISK = 'C:\Backups\MiBase_Log.trn'
WITH RECOVERY; --finalizar
```

COPY-ONLY BACKUP

```
BACKUP LOG MiBase
TO DISK = 'C:\Backups\MiBase_Log_CopyOnly.trn'
WITH COPY_ONLY;
--WITH COPY_ONLY → indica que el backup es independiente de la secuencia habitual.
```

RESTAURARLA

--se puede hacer como el full backup o igual para log backup

FILE / FILEGROUP BACKUP

--Permite hacer backup solo de archivos específicos o grupos de archivos

```
BACKUP DATABASE MiBase
FILE = 'MiArchivo1' -- o FILEGROUP = 'Primario', 'DataGroup1'
TO DISK = 'C:\Backups\MiBase_File.bak'
-- FILE = 'NombreArchivo' → indica qué archivo de la base de datos respaldar
-- FILEGROUP → indica los grupos de archivos a respaldar
```

RESTAURARLA

```
-- Restaurar solo un archivo específico
RESTORE DATABASE MiBase
FILE = 'MiArchivo1'
FROM DISK = 'C:\Backups\MiBase_File.bak'
WITH NORECOVERY;
```

```
-- Restaurar otro archivo si es necesario
RESTORE DATABASE MiBase
FILE = 'MiArchivo2'
FROM DISK = 'C:\Backups\MiBase_File2.bak'
WITH RECOVERY
```

SECUENCIAS

SINTAXIS BÁSICA PARA CREAR UNA SECUENCIA

```
CREATE SEQUENCE esquema.name_secuencia
AS INT
START WITH 1          -- valor inicial
INCREMENT BY 1        -- incremento
MINVALUE 1            -- valor mínimo
MAXVALUE 1000         -- valor máximo
CYCLE;                -- reinicia cuando llega al máximo
```

```
NEXT VALUE FOR NombreSecuencia --obtener el siguiente valor de la secuencia
```

```
-- tabla de secuencia
select * from sys.sequences
```

EJEMPLOS

```
-- creamos la secuencia
CREATE SEQUENCE SecuenciaFactura
AS INT
START WITH 1000
INCREMENT BY 1
go
```

```
--solo si no esta creada la tabla sobre la que vamos a itera
```

```
CREATE TABLE Facturas (
    IdFactura INT PRIMARY KEY,
    Cliente NVARCHAR(100),
    Total DECIMAL(10,2)
)
Go
```

```
-- ingresar los valores de la secuencias en la primaryKey de una table
INSERT INTO Facturas (IdFactura, Cliente, Total)
VALUES (NEXT VALUE FOR SecuenciaFactura, 'Juan Pérez', 150.75),
       (NEXT VALUE FOR SecuenciaFactura, 'Ana López', 299.99)
go
```

DIFERENCIA CON IDENTITY:

- IDENTITY está ligado a una tabla/columna.
- SEQUENCE es independiente y se puede usar en varias tablas o donde lo necesites

TRIGGERS

General

```
create trigger name_trigger -- creaar un trigger
on table -- especificamos sobre la tabla que se va a ejecutar
after update(delete, insert) -- tipo de trigger
as
    begin
        -- cuerpo de la accion
    End

drop trigger name_trigger -- eliminar un trigger
```

EJEMPLOS

Ejemplos_1:

Crea un trigger que se dispare después de una actualización en la tabla Agencia, y registre en AuditoriaCambiosSalario cualquier cambio en el campo Salario

```
create table registro ( --creamos la tabla donde iran los registros
registro_id int identity primary key,
cliente_id nvarchar(5),
usuario_id nvarchar(5),
)
go

create trigger ActualizacionAgencia
on Agencia
after update
as
begin

    declare @codigo_cliente nvarchar(5);
    -- mismo tipo de varianles que la tabla
    select @codigo_cliente = d.agencia_id from deleted d;
    -- asignamos un valor a la varibale
    print 'Se actualizo tal agencia: ' + @codigo_cliente;
    -- mensaje en consola

    insert into registro(usuario_id,cliente_id)
    -- ingresamos los valores a la tabla creada
    values (@codigo_cliente, user) -- por eso debe ser del mismo tipo XD

end
go

select * from registro -- confirmamos los registros
go

update Agencia -- probar si funciona el trigger
set descripcion = 'adios'
where Agencia.agencia_id = 110
go
```

EJERCICIOS

Con cursores

Para la base de datos NORTHWND cree una nueva tabla, que permita puntuar a los customersID con más órdenes un TOP 5, la nueva tabla deberá tener los siguientes campos CustomerID, Nombre, QuantityOrders, para realizar la inserción en la nueva tabla, deberá utilizar un cursor para contabilizar e insertar en la tabla

```
use Northwind
go
```

```
create table TopCustomers( -- 1) creamos la tabla
CustomerID NVARCHAR(5),
Nombre varchar (100),
QuantityOrders int
)
```

```
DECLARE @CustomerID NVARCHAR(5), -- 2) declaramos las variables
@Nombre NVARCHAR(100),
@QuantityOrders INT;
```

```
DECLARE customer_cursor CURSOR FOR -- 3) declaramos el cursor
```

```
select top(5) ord.CustomerID, cu.ContactName ,count (ord.OrderID) as n_ordenes
from Orders ord
inner join Customers cu on cu.CustomerID = ord.CustomerID
group by ord.CustomerID, cu.ContactName
ORDER BY COUNT(ord.OrderID) DESC;
```

-- 4) hacemos una especie de tabla temporal (no es, solo es un resultado)

```
OPEN customer_cursor; -- 5) abrimos el cursor que declaramos en (3)
```

```
FETCH NEXT FROM customer_cursor INTO @CustomerID, @Name, @QuantityOrders;
-- 6) iteramos en el resultado (supuesta tabla) y con INTO asignamos los valores
a las variables que creamos en (2)
```

```
WHILE @@FETCH_STATUS = 0 -- 7) recoremos la supuesta tabla de (4)
BEGIN
```

```
    INSERT INTO TopCustomers (CustomerID, Nombre, QuantityOrders)
    VALUES (@CustomerID, @Name, @QuantityOrders);
```

-- 8) ingresamos los vaores de las variables a la tabla creada en (1)

```
    FETCH NEXT FROM customer_cursor INTO @CustomerID, @Name, @QuantityOrders;
```

-- 9) recorremos la siguinete fila del resultado (4)

```
END
```

```
CLOSE customer_cursor; -- 10) cerramos el cursor que abrimos en (5)
```

```
DEALLOCATE customer_cursor; -- 11) borramos de memoria el resultado (4)
```

AUDITORIA DE BDs

```
--esto revisa los login que existena nivel servidor
SELECT * FROM sys.server_principals WHERE name = 'userDD1';

--Establece base de datos por defecto o modificar la base de datos a la que tiene
acceso
-- asi solo tendra accesos a la BD que definamos
ALTER LOGIN userDD1 WITH DEFAULT_DATABASE = Usuarios;

--usuarios creados a nivel de servidor
SELECT name, type_desc, is_disabled
FROM sys.server_principals
WHERE type IN ('S', 'U'); -- S=SQL Login, U=Windows Login

SELECT *
FROM sys.database_principals
WHERE type IN ('S', 'U', 'G'); -- Usuarios SQL, Windows, Roles

-- Roles disponibles en la base actual
SELECT name, type_desc
FROM sys.database_principals
WHERE type = 'R';

-- Qué usuarios pertenecen a qué roles
SELECT r.name , m.name
FROM sys.database_role_members drm
JOIN sys.database_principals r ON drm.role_principal_id = r.principal_id
JOIN sys.database_principals m ON drm.member_principal_id = m.principal_id;

-- Permisos de usuarios sobre tablas
SELECT dp.name AS Usuario, obj.name AS Objeto, perm.permission_name,
perm.state_desc
FROM sys.database_permissions perm
JOIN sys.objects obj ON perm.major_id = obj.object_id
JOIN sys.database_principals dp ON perm.grantee_principal_id = dp.principal_id
ORDER BY Usuario, Objeto;

--Ver esquemas
SELECT name AS SchemaName, schema_id, principal_id
FROM sys.schemas;

-- Permisos a nivel de base de datos
SELECT dp.name, dp.type_desc , p.permission_name, p.state_desc
FROM sys.database_principals dp
LEFT JOIN sys.database_permissions p ON p.grantee_principal_id = dp.principal_id
WHERE dp.type IN ('S', 'U', 'G') ORDER BY dp.name;

-- Permisos a nivel de servidor
SELECT sp.name, sp.type_desc, perm.permission_name, perm.state_desc
FROM sys.server_principals sp
LEFT JOIN sys.server_permissions perm ON perm.grantee_principal_id =
sp.principal_id;

-- Triggers a nivel de base de datos
SELECT name, parent_class_desc, create_date, modify_date, is_disabled
```

```

FROM sys.triggers;

-- Triggers a nivel de servidor
SELECT name, parent_class_desc, create_date, modify_date, is_disabled
FROM sys.server_triggers;

-- Listado de bases de datos y sus configuraciones principales
SELECT name, database_id, create_date, compatibility_level, collation_name,
state_desc, recovery_model_desc
FROM sys.databases;

SELECT *
FROM sys.databases;

-- Listar todas las tablas
SELECT t.name AS Tabla, s.name AS Esquema, t.create_date
FROM sys.tables t
JOIN sys.schemas s ON t.schema_id = s.schema_id;

SELECT *
FROM sys.tables

-- Columnas de cada tabla
SELECT t.name AS Tabla, c.name AS Columna, ty.name AS Tipo, c.max_length,
c.is_nullable
FROM sys.columns c
JOIN sys.tables t ON c.object_id = t.object_id
JOIN sys.types ty ON c.user_type_id = ty.user_type_id
ORDER BY t.name;

-- Procedimientos almacenados
SELECT name, create_date, modify_date FROM sys.procedures;

-- Funciones
SELECT name, type_desc, create_date FROM sys.objects
WHERE type IN ('FN', 'TF', 'IF'); -- Escalares, tabla, inline

-- Vistas
SELECT name, create_date FROM sys.views;

-- Cuentas de SQL deshabilitadas o bloqueadas
SELECT name, is_disabled
FROM sys.sql_logins
WHERE is_disabled = 1;

-- Usuarios huérfanos (logins sin user en DB o viceversa)
EXEC sp_change_users_login 'Report';

--Dependencias (qué llama a qué)
-- Qué objetos dependen de otros (ejemplo: SP que usan tablas)
SELECT referencing_object_name = o1.name,
referencing_object_type = o1.type_desc,
referenced_object_name = o2.name,
referenced_object_type = o2.type_desc
FROM sys.sql_expression_dependencies d
JOIN sys.objects o1 ON d.referencing_id = o1.object_id
JOIN sys.objects o2 ON d.referenced_id = o2.object_id;

```

```

--Últimos inicios de sesión fallidos (si tienes login auditing activado)Revisar
en el Error Log:
EXEC xp_readerrorlog 0, 1
EXEC xp_readerrorlog 0, 1, '18456', NULL, NULL, NULL, 'DESC';
EXEC xp_readerrorlog 0, 1, 'Login failed', NULL, NULL, NULL, 'DESC'

--0 = Archivo de error actual (0=actual, 1=anterior, etc.)
--1 = Tipo de log (1=SQL Server, 2=SQL Agent)
--'18456' = Texto a buscar
--NULL = Segundo texto a buscar (opcional)
--NULL, NULL = Fechas (opcional)

-- Intentos fallidos
EXEC xp_readerrorlog 0, 1, NULL, NULL, NULL

-- Logins exitosos
EXEC xp_readerrorlog 0, 1, Login_succeeded, NULL, NULL, NULL;

-- Todos los eventos de autenticación
EXEC xp_readerrorlog 0, 1, Login, NULL, NULL, NULL, 'DESC';
EXEC xp_readerrorlog 0, 1, 'Login succeeded for user'', NULL, NULL, NULL, 'DESC';

-- Método más robusto
CREATE TABLE #Logins (
    LogDate datetime,
    ProcessInfo varchar(100),
    Text varchar(4000)
);

INSERT INTO #Logins
EXEC xp_readerrorlog 0, 1, NULL, NULL, NULL, NULL, 'DESC';

SELECT * FROM #Logins
WHERE Text LIKE '%Login%succeeded%' OR Text LIKE '%Login%failed%'
ORDER BY LogDate DESC;

DROP TABLE #Logins;

-- ver los logins de los usuarios y a que hora
SELECT login_name, host_name, program_name, login_time
FROM sys.dm_exec_sessions
WHERE login_name in ('UserDD1', 'Usr_Richard')

```

METADATOS

```

-- obtener metada de los objetos
SELECT
    OBJECT_ID('Agencia') AS ID_Objeto,
    OBJECT_NAME(OBJECT_ID('Agencia')) AS Nombre_Objeto,
    TYPE_NAME(OBJECTPROPERTY(OBJECT_ID('Agencia'), 'BaseType')) AS Tipo

```

TEMAS POR CUBRIR

✓ **Backups y recuperación (Disaster Recovery)**

- ❖ Integración con Python, PowerShell o herramientas de monitoreo (backups)
- ❖ Backups en la nube (Azure, AWS, etc.).
- ❖ Alta disponibilidad y replicación (AlwaysOn, Log Shipping, Mirroring, Clustering).
- ❖ Monitoreo de rendimiento (tuning de queries, índices, planes de ejecución).
- ❖ Automatización con SQL Server Agent (jobs, alertas, mantenimiento).
- ❖ Auditoría y seguridad avanzada.

Bloque 2: Administración intermedia (lo que debes aprender ahora)

- ❖ Backups diferenciales, completos, transaccionales.
- ❖ Restauración en diferentes escenarios (point-in-time recovery).
- ❖ SQL Server Agent: automatización de respaldos, limpiezas, envíos de correos.
- ❖ Índices: tipos de índices, cuándo usarlos, mantenimiento.
- ❖ Monitoreo de performance con DMVs (Dynamic Management Views)

Bloque 3: Administración avanzada (para entornos empresariales)

- ❖ Replicación, AlwaysOn, Log Shipping.
- ❖ Auditoría y cifrado de datos.
- ❖ Partitioning y compresión de tablas.
- ❖ Optimización de queries con planes de ejecución.
- ❖ Diseño de arquitecturas de alta disponibilidad

Auditoría de base de datos

- ❖ Implementar un log que registre quién inserta/actualiza/elimina datos.
- ❖ Usar triggers y tablas de auditoría.

Sistema de respaldo y recuperación

- ❖ Crear scripts que hagan backups automáticos diarios.
- ❖ Restaurar en otro servidor de pruebas.

Dashboard de rendimiento

- ❖ Usar DMVs para medir queries más lentas, índices usados/no usados, sesiones activas.
- ❖ Guardar en tablas históricas y mostrar en Power BI/Excel.

Alta disponibilidad en un laboratorio

- ❖ Configurar AlwaysOn en dos instancias de SQL Server con máquinas virtuales.
- ❖ Simular caídas y ver recuperación automática.

Orden sugerido de estudio inmediato

- ❖ **Backups y restauración**
- ❖ Automatización con Jobs

- ❖ Índices y performance básico
- ❖ Monitoreo y seguridad avanzada
- ❖ Alta disponibilidad (laboratorio)

PROYECTOS POR CUBRIR

BACKUP

- ❖ Restaurar un backup completo.
 - ❖ Restaurar backups en cadena (completo + diferencial + log).
 - ❖ Restaurar hasta un punto en el tiempo.
 - ❖ Automatizar backups con SQL Server Agent.
- **Proyecto:** crea un plan de respaldo automatizado (ejemplo: respaldo completo cada domingo, diferenciales diarios, logs cada hora). Documenta cómo restaurar en distintos escenarios

SQL Server Agent y Jobs (automatización)

- ❖ Crear jobs programados (respaldos, limpieza de logs, envío de reportes).
 - ❖ Crear alertas para errores críticos (por ejemplo, si falla un backup).
 - ❖ Monitoreo de ejecución de jobs.
- **Proyecto:** automatiza un job que respalde y otro que elimine logs viejos

Índices y optimización de consultas (Performance Tuning básico)

- ❖ Tipos de índices: clustered, non-clustered, unique, covering, filtered.
 - ❖ Cómo leer un plan de ejecución.
 - ❖ Detectar consultas lentas con DMVs.
 - ❖ Uso de STATISTICS para ver cómo SQL Server decide el plan.
- **Proyecto:** crea un sistema con tablas grandes, ejecuta consultas sin índices y con índices, mide la diferencia con SET STATISTICS IO/TIME

Monitoreo y seguridad avanzada

- ❖ Monitoreo con DMVs
 - Consultas más costosas.
 - Índices usados/no usados.
 - Sesiones bloqueadas.
 - ❖ Seguridad
 - Roles fijos de servidor vs. roles personalizados.
 - Permisos a nivel de columna.
 - Auditoría con triggers o SQL Server Audit.
- **Proyecto:** implementa una tabla de auditoría que registre usuario, fecha, acción en cada UPDATE/DELETE.

Alta disponibilidad y recuperación avanzada (cuando domines lo anterior)

- ❖ Log Shipping.

- ❖ Replicación.
 - ❖ AlwaysOn Availability Groups.
 - ❖ Mirroring (legacy, pero bueno conocerlo).
- **Proyecto:** montar un laboratorio con 2 máquinas virtuales y probar un failover.

Python como aliado en tu camino (explicado en cada punto)

Python es muy poderoso porque conecta perfecto con **bases de datos, automatización y seguridad**. Veamos cómo:

1 Automatización

Con Python puedes crear scripts para **automatizar tareas repetitivas** de un DBA o sysadmin.

- Script que haga **backups automáticos** de la base de datos cada noche.
- Script que **revise logs de errores** y te mande un correo/alerta.
- Automatizar la creación de **usuarios y roles** en una BD.

Trabajar con bases de datos

Python se conecta fácilmente con **MySQL, PostgreSQL, Oracle, SQL Server, MongoDB** y más.

- Con librerías como **SQLAlchemy, psycopg2, pyodbc**,.Esto es oro para un DBA, porque puedes **administrar y analizar bases de datos con código**.

Análisis de logs

Los **logs** son archivos que guardan todo lo que pasa en un sistema o BD (errores, accesos, consultas, intentos de login).

Python puede leer esos archivos y:

Buscar intentos de **hackeo o accesos sospechosos**.

- Detectar errores frecuentes.
- Resumir actividad de usuarios.
- Esto conecta tu rol de **DBA con ciberseguridad**.

ETL (Extract, Transform, Load)

Python para ETL maneja datos de muchos formatos (CSV, JSON, APIs, BD, Excel).

- **Extract:** sacar datos de una BD de ventas.
- **Transform:** limpiar duplicados, convertir fechas.
- **Load:** cargar en un Data Warehouse para BI.

Aquí se conecta con **Business Intelligenc**

Lo que estamos haciendo hoy

Módulo 1 – Introducción a SQL Server Agent

Aprende:

Qué son los **Jobs** (tareas programadas).

Qué son los **Steps** (pasos dentro de un Job).

Qué son los **Schedules** (horarios para ejecutar).

Ejercicio:

Verifica que el servicio **SQL Server Agent** esté iniciado en tu servidor.

Módulo 2 – Crear tu primer Job

En SSMS → **SQL Server Agent** → **Jobs** → **New Job**.

Ponle un nombre: Backup_Full_MiBase.

Agrega un **Step** con este código:

```
BACKUP DATABASE MiBaseTO DISK = 'C:\Backups\MiBase_Full.bak'WITH INIT;
```

Crea un **Schedule** para ejecutarlo todos los días a las 12:00 am.

Ejercicio: Ejecuta el job manualmente y revisa el archivo .bak.

Módulo 3 – Jobs con múltiples Steps

Objetivo: automatizar backups completos.

Crea un job con **varios Steps**:

Step 1: Backup Full (noche).

Step 2: Backup Differential (cada 4 horas).

Step 3: Backup Log (cada 15 min).

Ejemplo de código para el Log:

```
BACKUP LOG MiBaseTO DISK = 'C:\Backups\MiBase_Log.trn';
```

Ejercicio: programa los tres y revisa que se creen en los horarios.

Módulo 4 – Monitoreo y Alertas

Objetivo: aprender a detectar errores en Jobs.

Revisa el historial de ejecución de un Job.

Configura notificaciones:

Enviar correo si un backup falla (requiere Database Mail configurado).

O revisar manualmente en SSMS → SQL Server Agent → Job History.

Ejercicio: provoca un error (ej. cambia la ruta de backup a una carpeta inexistente) y verifica la alerta.

Módulo 5 – Buenas Prácticas

Objetivo: aplicar lo que hacen los DBAs en empresas reales.

Guarda backups en un **disco distinto** al de SQL Server.

Usa nombres dinámicos con fecha en los backups:

```
DECLARE @FileName NVARCHAR(200); SET @FileName = 'C:\Backups\MiBase_Full_' +  
        CONVERT(VARCHAR(10), GETDATE(), 112) + '.bak';
```

```
BACKUP DATABASE MiBase TO DISK = @FileName WITH INIT;
```

Programa limpieza de backups viejos (ej. mantener solo 7 días).

Documenta tu estrategia de backups.

Módulo 6 – Proyecto Final

Objetivo: simular lo que harías como DBA en una empresa.

Crea una estrategia de backups automática para MiBase:

Full Backup → diario a medianoche.

Differential → cada 4 horas

Transaction Log → cada 15 min.

Monitorea los jobs una semana y revisa los archivos generados.
Haz una **prueba de restauración** con los backups creados.

REQUISITOS PARA TRABAJOS

- Implementar/aplicar políticas de auditoría y seguridad en Bases de Datos.
 - Aseguramiento de la integridad de los datos
 - Brindar soporte para configuración e implementación de componentes de Bases de Datos.
 - Velar por la continuidad y disponibilidad de todos los sistemas de información productivos de la Institución.
 - Definir y Asegurar esquema de seguridad, permisos, perfiles, políticas, cuotas, límites, etc. Control de la evolución de espacio en bases de datos.
 - Atender Incidentes o tickets de servicio relacionados a Infraestructura o Base de datos a través de mesa de servicio
-
- Oracle SOA Suite, Weblogic, Application Server.
 - Conocimientos de administración de herramientas de Business Intelligence.
 - Conocimientos de manejo de respaldo de la información y procesos de recuperación de desastres.
 - Sistemas Operativos: Windows. Linux, Unix / Medio.
 - Conocimiento avanzado en manejadores de Base de Datos: Oracle, SQL Server.
 - Conocimiento intermedio de la metodología ITIL.
 - Conocimientos de herramientas de integración y transformación de Datos.
 - Conocimientos de TABLEAU.
-
- Conocimientos de DMBOK.
 - Lenguaje SQL en nivel avanzado. (DML, DDL, OLTP, DCL, TCL, Logon)
 - Procesos ETLs en nivel intermedio.
 - Diseño y Modelado de Data Warehouse en nivel intermedio.
 - Procesamientos de datos (Batch, Streaming, API, etc) en nivel intermedio.
 - Lenguaje Python en nivel avanzado.
 - Diseño y Modelado de Datalake / Lakehouse en nivel intermedio.
 - Conocimiento conceptual de Gobernanza, seguridad y calidad de datos.
 - Azure DevOps (Control de Versiones e Integración Continua) en nivel avanzado.
 - Compresión de la Nube y de Servicios de Ingesta de Datos.
 - Conocimiento de la herramienta de Portal de Azure (Cloud).

Certificaciones

- DP 203
- Databricks Lakehouse Fundamentals
- AI 900
- AZ900
- DP900